Future Intel Internship — Task 1 Detailed Report

Author: Future Intern
Date: September 12, 2025

A comprehensive report documenting the discovery, evidence, impact, and remediation recommendations for issues found during web application testing of OWASP Juice Shop (local instance). This narrative-style report includes technical details, business impact, exploit scenarios, and suggested mitigation plans.

Table of Contents

1. Executive Summary
2. Scope, Environment & Tools
3. Methodology
4. Findings
4.1 Error Handling — Exposed Stack Trace
4.2 Security Question Dropdown — Blocked UX & Risks
5. Detailed Impact & Consequences
6. Remediation & Mitigation Plan
7. Evidence & Artifacts
8. Appendices & References

1. Executive Summary

During Task 1 of the Future Intel internship, the OWASP Juice Shop application was analyzed in a controlled local environment. Two related issues were documented: (1) improper error handling that exposes internal stack traces when malformed input is submitted, and (2) a fragile registration UX where the security question dropdown failed to populate when HTTP traffic was intercepted, blocking user registration flows. This report provides evidence, technical analysis, likely exploitation scenarios, business impacts, and prioritized remediation steps.

2. Scope, Environment & Tools

Scope: Local deployment of OWASP Juice Shop accessible at http://192.168.1.30:3000. Environment: Kali Linux VM (VirtualBox host), Node.js runtime, Burp Suite acting as an intercepting proxy. Tools used: Burp Suite (Proxy, Repeater), Firefox (with proxy configured), curl, npm, node, and built-in developer tools. All testing was performed on the local training instance and not against any external or production systems.

3. Methodology

The approach combined manual reconnaissance, proxy interception, and targeted malformed inputs to observe server behaviour. Key steps included: Mapping visible endpoints via browser interactions and Burp site map. Intercepting registration and login POST requests and replaying/mutating them in Burp Repeater. Submitting syntactically invalid JSON to trigger parser errors (controlled fault injection) to observe server responses. Collecting raw request/response artifacts and screenshots for reproducibility.

4. Findings

4.1 Error Handling — Exposed Stack Trace

Summary:

A malformed JSON body (missing the password value) was submitted to the login endpoint. The server returned an HTTP 500 response containing an 'error' JSON object with a human-readable message and a full stack trace. The stack trace included file paths and line numbers within the application source and Node.js modules.

Exact request payload used (PoC):

```
{ "email": "test@example.com", "password": }
```

Server response (summary): HTTP/1.1 500 Internal Server Error with JSON body containing 'message' and 'stack' properties. The 'stack' lists functions and file locations inside the application.

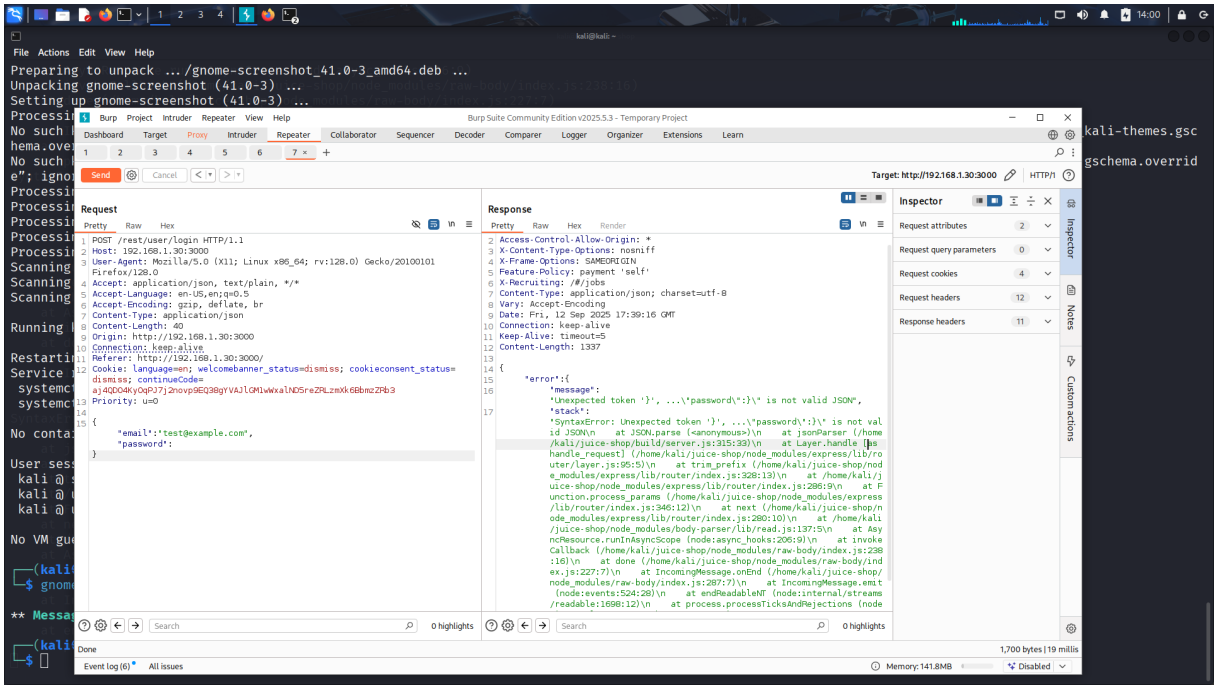

Figure 1 — Burp Repeater showing the malformed JSON request (left) and server stack trace response (right).

4.2 Security Question Dropdown — Blocked UX & Risks

Summary:

When Burp's interception was enabled and the GET request to /api/SecurityQuestions/ was left intercepted (not forwarded), the client could not populate the security question dropdown. This blocked registration. With interception off, the request completed and the dropdown populated as expected. The observed behaviour demonstrates fragile client-server coupling and unhandled conditions on the client when required data is missing.

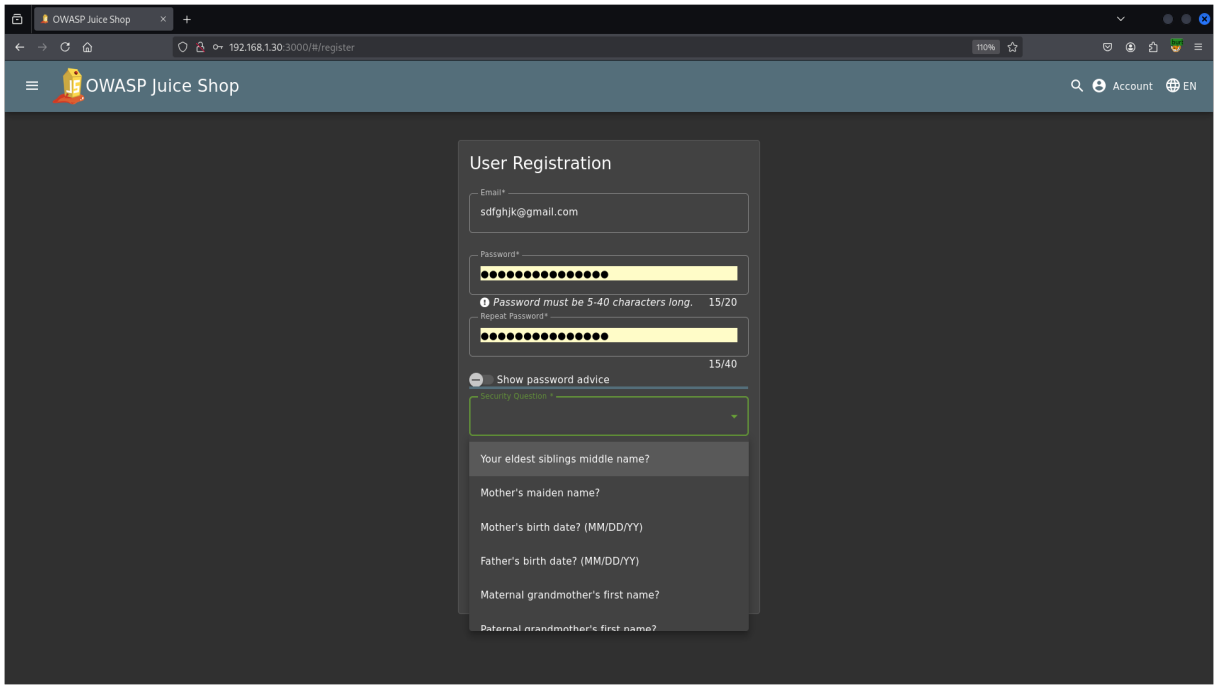

Figure 2 — Security question dropdown populated after interception was disabled.

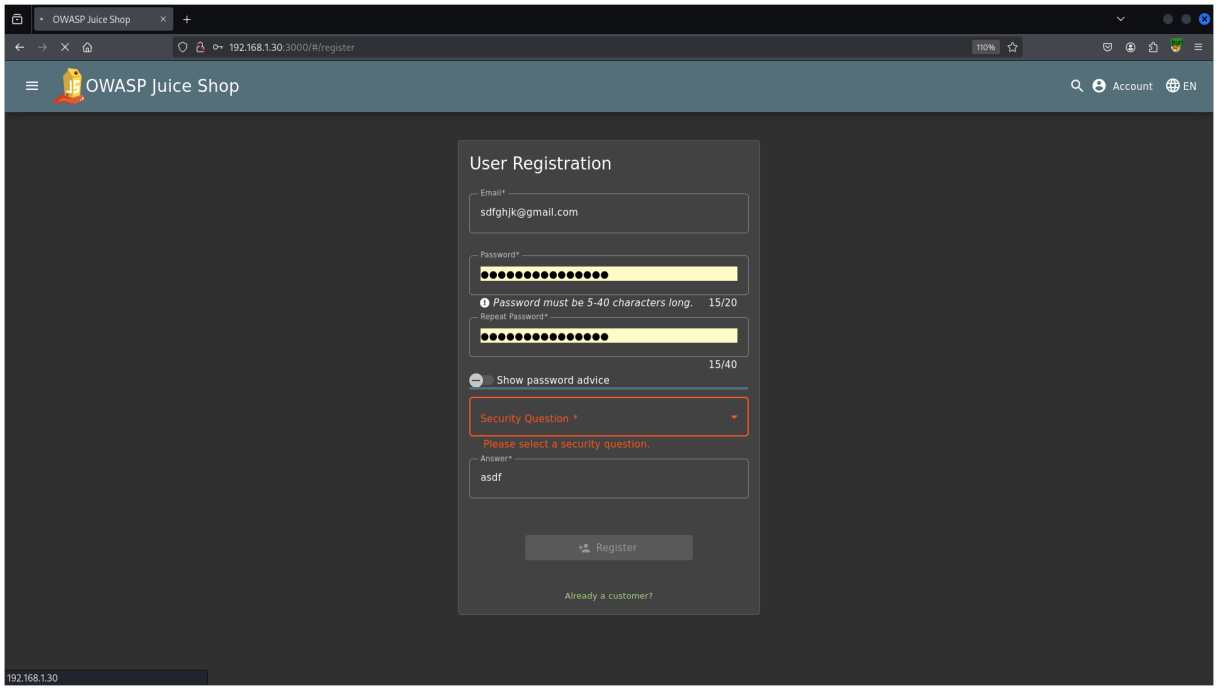

Figure 3 — Security question dropdown list visible.

5. Detailed Impact & Consequences

5.1 Error Handling — Technical Consequences

Targeted exploitation acceleration: Stack traces provide precise fingerprints for frameworks, modules and code paths. Attackers can quickly match versions to public CVEs and expedite exploit development. **Attack surface expansion:** Knowledge of internal endpoints, admin routes or file locations reduces reconnaissance time and allows attackers to chain vulnerabilities (e.g., leveraging an information leak to craft injection payloads). **Privilege escalation:** Exposed module names and functions can reveal authentication logic and potential unsafe authority checks that enable privilege escalation. **Automated weaponization:** Attackers can build automated scanners and exploit tools that target specific functions revealed by the stack trace, increasing scale and speed of attacks.

5.2 Error Handling — Business & Operational Consequences

Data breach potential: Successful exploitation informed by stack traces may lead to database access and PII exposure, resulting in regulatory fines and user notification obligations. **Incident response costs:** Time spent investigating, containing, and remediating incidents raises operational costs and drains engineering resources. **Customer trust erosion:** Publicized breaches or visible application crashes damage brand reputation and may lead to user churn. **Legal and compliance exposure:** Depending on data types exposed (emails, payments), the organization may face GDPR/CCPA penalties or contractual liabilities.

5.3 Security Question — Technical Consequences

Registration availability failure: Blocking a required API call prevents legitimate users from registering. **Bypass & account takeover risk:** If the server accepts client-provided security question values without server-side validation, attackers could manipulate answers to perform account recovery hijacks. **Bot-driven abuse:** Fragile validation may be exploited by scripts to create large numbers of accounts for spam, fraud, or to influence platform metrics.

5.4 Security Question — Business & Operational Consequences

Loss of conversions: Broken sign-up UX directly impacts new user registrations and growth. **Support burden:** Increased user complaints and support tickets for account creation failures. **Fraud exposure:** Automated account creation can be used to facilitate spam, fake reviews, or fraudulent transactions.

6. Remediation & Mitigation Plan

We recommend the following prioritized actions (short-term, medium-term, long-term) to address both findings and reduce risk exposure.

Priority	Action	Notes
P0 (Immediate)	Remove stack trace from HTTP responses; add global error handler	Log full errors internally; return generic messages
P1 (Short-term)	Validate JSON schema on API inputs; reject malformed payloads with 4xx	Use JOI or Yup (or express-validator)
P2 (Short-term)	Ensure client handles missing dropdown data gracefully; add retry/fallback	Provide cached defaults or disable form until data
P3 (Medium)	Rate-limit registration endpoints; add bot protection	Consider CAPTCHA or device fingerprinting
P4 (Long-term)	Automated monitoring and alerting for repeated 5xx errors	Integrate with SIEM/logging

Implementation notes:

- Add Express error middleware that catches errors and sends a generic JSON payload: {"error":"Internal server error"}. - Use JSON schema validation as early as possible in request handling to reject malformed payloads before parsing. - Harden registration logic: always validate security question IDs server-side and verify answers against stored hashes. - Add monitoring and alerting for unusual 5xx spikes and failed registration rates.

7. Evidence & Artifacts

Included artifacts for reproducibility and audit:
• pocs/error-handling-request.txt — raw POST request (malformed JSON)
• pocs/error-handling-response.txt — raw 500 response containing stack trace
• screens/step-03_server-response.png — screenshot of Repeater showing stack trace
• FutureIntel_Task1_Report_Detailed.pdf — this document

8. Appendices & References

- OWASP Top Ten — https://owasp.org/www-project-top-ten/ - OWASP Web Security Testing Guide — https://owasp.org/www-project-web-security-testing-guide/ - Express error handling documentation - Juice Shop challenge documentation and solutions (local lab)

End of report.